Mini Project: Definition of the Problem

Name: Swastik Anurag Vyas

Registration No.: 2430030281

In collaboration with: Rudra Pratap Singh 2430030327

Waybill — courier delivery route optimisation

Waybill addresses the daily routing decision of a courier company. Given a depot, a set of customer deliveries, and the cost of travelling between every pair of locations, the system must produce routes that serve all customers at minimum total travel cost. A route starts at the depot, visits its assigned customers once, and returns to the depot. The output must also be feasible: it cannot exceed a vehicle's capacity, violate a customer's delivery window, or run beyond the driver's shift.

The project has two deliberately connected levels. Its course-algorithm core is the Travelling Salesperson Problem (TSP): one vehicle must visit every customer exactly once and return to the depot. This gives a precise setting for greedy, exhaustive, dynamic-programming, and branch-and-bound algorithms, and permits optimal routes to be proved on small instances. The practical extension is the Capacitated Vehicle Routing Problem with Time Windows (CVRPTW), where deliveries are divided among a fixed fleet. TSP is therefore the one-vehicle, unlimited-capacity, no-time-window special case of the same model, not a separate unrelated application.

Road costs may be asymmetric: travelling from location i to location j can differ from travelling from j to i because of one-way roads. All costs and times are stored as non-negative integers (metres, seconds, or TSPLIB distance units), avoiding rounding ambiguity in comparison and feasibility checks. The baseline problem uses static road conditions for one planning run; live traffic, split deliveries, multiple depots, and pickups are outside this mini-project.

Mathematical representation

Let the delivery locations form a directed complete graph $G = (V, E)$, where $V = \{0, 1, \ldots, n-1\}$. Node r is the depot and $C = V - \{r\}$ is the set of customers. The cost matrix $D = [d_{ij}]$ gives the non-negative cost of travelling from node i to node j. No symmetry is assumed: d_{ij} need not equal d_{ji}.

For the TSP core, a feasible tour is $P = (r, v_1, v_2, \ldots, v_{n-1}, r)$,

where $(v_1, \ldots, v_{n-1})$ is a permutation of C. Its cost is

$$L(P) = d(r, v_1) + \sum_{h=1}^{n-2} d(v_h, v_{h+1}) + d(v_{n-1}, r).$$

The required output is a tour P^* such that

$$P^* = \arg\min \{ L(P) : P \text{ visits every customer exactly once} \}.$$

together with its total cost. An exact solver may label the result optimal only after it has completed its proof condition; a heuristic result is instead reported as feasible, with its method and any known comparison stated clearly.

For the CVRPTW extension, let K be the available vehicles. Each customer i in C has parcel demand q_i, service time s_i, and a closed delivery window $[a_i, b_i]$. Each vehicle k in K has capacity Q_k, shift $[A_k, B_k]$, and optionally a maximum route duration. The input also contains a travel-time matrix $T = [t_{ij}]$, in seconds, when cost and travel time are different measures. A solution is a set of routes $R_k = (r, \ldots, r)$, with at most one route for each vehicle. It is feasible when every customer belongs to exactly one route, each route satisfies $\sum_{i \in R_k} q_i \le Q_k$, and its service schedule reaches each customer within $[a_i, b_i]$ and returns within the vehicle shift. Early arrival is allowed and creates waiting time. Among all feasible route sets, the objective is

minimize total cost = sum[k in K] sum[(i, j) consecutive in R_k] d_ij.

The output is the ordered customer sequence for every used vehicle, the total cost, route schedules and loads, and a feasibility status. If a complete feasible plan cannot be built, unserved customers and the reason are returned explicitly rather than presenting a partial route as a valid answer.

Input, output, and possible data structures

Item	Representation	Purpose
Stops	Immutable array/tuple of records: ID, coordinates, demand, time window, and service time	Keeps domain IDs separate from algorithm indices; index r identifies the depot.
Travel cost/time	Flat n × n integer matrix, accessed as matrix[i*n + j]	Gives constant-time directed-cost lookup and is practical for dense routing graphs.
Vehicles	Array/tuple of records: ID, capacity, shift bounds, and maximum duration	Defines the fleet constraints for CVRPTW.
TSP candidate	Ordered array of customer indices, with the depot implicit at both ends	Compact tour representation; its cost is scored by adjacent matrix lookups.
Multi-vehicle solution	Array/tuple of Route(vehicle_id, customer_sequence) records plus status and total cost	Records the assignment and order of deliveries without duplicating the depot.
Exact-search state	Bit set of visited customers, current node, partial cost, and predecessor/parent pointer	Supports Held–Karp dynamic programming and Branch-and-Bound while detecting repeated visits efficiently.
Branch-and-Bound frontier	Min-priority queue keyed by a valid lower bound	Expands the most promising partial route first and prunes states that cannot beat the current solution.

This representation lets Waybill use the same validated instance data for the TSP algorithms required by the course and for the realistic CVRPTW extension.